Results

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with eachother. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

GPU Configuration

Parameter	Value
GPU	NVIDIA GeForce RTX 4060 Ti
Compute Capability	8.9
Number of SMs	34
Max threads per SM	1536
Max blocks per SM	24
Warp size	32
BLOCK_SIZE	64
GRID_SIZE	816
Vector size	40 MB
Number of float elements	10,485,760

Console Output

```
================================================================
GPU Configuration Information
================================================================
GPU Device: NVIDIA GeForce RTX 4060 Ti
Compute Capability: 8.9
Number of SMs: 34
Max threads per SM: 1536
Max blocks per SM: 24
Warp size: 32

Optimal Launch Configuration:
BLOCK_SIZE: 64 (threads per block)
GRID_SIZE: 816 (number of blocks)

Vector Info:
Vector size: 40 MB
Number of float elements: 10485760
================================================================

================================================================
Task 1: CPU Reduction (Baseline)
================================================================
```

```
CPU Result: 10485760.00 (expected: 10485760)
CPU Average Time: 22.4195 ms
======================================================================

======================================================================
Task 2: GPU Atomic Cascaded Reduction (from HW7)
======================================================================
Launch Config: <<<816, 64>>>
Reductions match.

GPU Atomic Cascaded Average Time: 0.2176 ms
======================================================================

======================================================================
Task 3: Harris Cascaded with threadfence (All Optimizations)
======================================================================
Launch Config: <<<816, 64>>>
Optimizations applied:
  - Grid-stride loop (thread coarsening)
  - Warp shuffle reduction (no shared memory bank conflicts)
  - Block-level reduction with minimal synchronization
  - __threadfence() for inter-block communication
  - Single kernel launch (no second kernel needed)
Reductions match.

GPU Harris Cascaded Average Time: 0.2343 ms
======================================================================

======================================================================
Task 4: Harris Cascaded Optimized (with loop unrolling)
======================================================================
Launch Config: <<<816, 64>>>
Additional optimization: Loop unrolling (4x)
Reductions match.

GPU Harris Cascaded Optimized Average Time: 0.2391 ms
======================================================================
```

Performance Results

Task	Description	Avg Time (ms)	Status	Speedup vs CPU
Task 1	CPU Reduction (Baseline)	22.4195	–	1.0x
Task 2	GPU Atomic Cascaded (HW7)	0.2176	Reductions match	~103x
Task 3	Harris Cascaded with threadfence	0.2343	Reductions match	~96x

Task	Description	Avg Time (ms)	Status	Speedup vs CPU
Task 4	Harris Cascaded Optimized (loop unroll)	0.2391	Reductions match	~94x

Analysis

Performance Ranking (Fastest to Slowest):

1. **GPU Atomic Cascaded (HW7)** - 0.2176 ms (~103x speedup)
2. **Harris Cascaded with threadfence** - 0.2343 ms (~96x speedup)
3. **Harris Cascaded Optimized** - 0.2391 ms (~94x speedup)
4. **CPU Reduction** - 22.4195 ms (baseline)

Key Observations:

1. **All GPU implementations are significantly faster than CPU** (~100x speedup), demonstrating the massive parallelism advantage for reduction operations.
2. **Atomic Cascaded slightly outperforms Harris Cascaded**: This is an interesting result. The atomic cascaded implementation from HW7 performs marginally better (0.2176 ms vs 0.2343 ms). This can be explained by:
   - The RTX 4060 Ti (Compute Capability 8.9) has highly optimized hardware atomic units
   - The overhead of `__threadfence()` and the atomic counter management adds slight latency
   - With only 816 blocks, the number of atomic operations is already minimal
3. **Loop unrolling did not improve performance**: The additional loop unrolling (4x) actually slightly increased execution time. This suggests:
   - Register pressure may have increased
   - The compiler already performs aggressive optimizations
   - The memory bandwidth is the bottleneck, not arithmetic operations
4. **All GPU implementations produce correct results**: The reductions match the CPU baseline (10,485,760, as expected when summing 10,485,760 floats each initialized to 1.0).

Why Atomic Cascaded Won:

On modern GPUs like the RTX 4060 Ti, the atomic hardware is extremely efficient. The cascaded approach already minimizes atomics to one per block, which is 816 operations. The `__threadfence()` approach adds: - Memory fence overhead - Atomic counter operations - Conditional branching for last-block detection

For this relatively small number of blocks, these overheads outweigh any theoretical benefit from avoiding atomics in the final reduction stage.

Conclusion:

Both GPU implementations achieve excellent performance (~100x speedup over CPU). The Harris cascaded algorithm with `__threadfence()` provides a valid alternative approach that avoids multiple atomic operations, but on modern hardware with efficient atomic units, the simpler atomic cascaded approach performs equally well or better for this problem size.